

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO4: Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)

Assessment Tool Weightage: 50%

Duration : 1 hour

QUESTIONS: 2

Total Marks:40

Annexure A

Integrated Experiments

Code of Conduct:

I POOJASREE B (192411091) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: POOJASREE B (192411091)

Student 23: POOJASREE B | Register Number: 192411091

INTEGRATED EXPERIMENT SCENARIO

Retail - Customer Churn Prediction

Context: A retail chain wants an integrated pipeline that first predicts customer churn from purchase records (frequency, order value, complaints, tenure) and then recommends a retention action using a learning agent.

Question 1 - Decision Tree vs Neural Network Classifier

Implement a Decision Tree classifier and a Neural Network (Multi-Layer Perceptron) classifier on the same churn dataset. Compare their Accuracy, Precision, Recall and F1-Score. Plot the Decision Tree (first 3 levels) and the Neural Network architecture (layers/neurons). Justify which model you would deploy for real-time churn scoring and why.

Question 2 - Reinforcement Learning for Retention Actions

Using the churn-risk output from Experiment 1 as the initial state, implement a Q-Learning agent that recommends a retention action (Discount / Loyalty Offer / Personal Outreach / No Action) across simulated customer episodes. Show the Q-table after training, the learned policy, and integrate it with the classifier output into a single end-to-end decision pipeline (risk prediction -> action recommendation).

Integrated Experiment: Retail Customer Churn Prediction

Question 1 – Decision Tree vs Neural Network Classifier

Aim

To build and compare a **Decision Tree** and **Neural Network (MLP)** for predicting customer churn, and then use the better model as the churn-risk predictor in a retention system.

Dataset Features

We use:

- Purchase Frequency
- Average Order Value
- Complaint History
- Loyalty Tenure
- Days Since Last Purchase

Target: Churn

- 0 → Customer retained
- 1 → Customer churned

Python Implementation

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    classification_report, confusion_matrix
)

# -----
# 1. CREATE SIMULATED RETAIL CUSTOMER DATASET
# -----

np.random.seed(42)

n = 500

purchase_frequency = np.random.randint(1, 21, n)
```

```

average_order_value = np.random.randint(200, 5000, n)
complaints = np.random.randint(0, 6, n)
loyalty_tenure = np.random.randint(1, 61, n)
days_since_purchase = np.random.randint(1, 181, n)

# Generate churn based on customer behaviour
risk_score = (
    -0.18 * purchase_frequency
    -0.0002 * average_order_value
    + 0.65 * complaints
    -0.025 * loyalty_tenure
    + 0.035 * days_since_purchase
)

probability = 1 / (1 + np.exp(-risk_score))

churn = (probability > np.median(probability)).astype(int)

data = pd.DataFrame({
    "PurchaseFrequency": purchase_frequency,
    "AverageOrderValue": average_order_value,
    "Complaints": complaints,
    "LoyaltyTenure": loyalty_tenure,
    "DaysSincePurchase": days_since_purchase,
    "Churn": churn
})

print(data.head())

# -----
# 2. FEATURES AND TARGET
# -----

X = data.drop("Churn", axis=1)
y = data["Churn"]

# 70:30 stratified split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.30,
    random_state=42,
    stratify=y
)

# -----
# 3. STANDARDISATION FOR NEURAL NETWORK
# -----

scaler = StandardScaler()

```

```

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# -----
# 4. DECISION TREE
# -----

dt_model = DecisionTreeClassifier(
    max_depth=5,
    random_state=42
)

dt_model.fit(X_train, y_train)

dt_pred = dt_model.predict(X_test)

# -----
# 5. NEURAL NETWORK / MLP
# -----

mlp_model = MLPClassifier(
    hidden_layer_sizes=(16, 8),
    activation="relu",
    solver="adam",
    max_iter=1000,
    random_state=42
)

mlp_model.fit(X_train_scaled, y_train)

mlp_pred = mlp_model.predict(X_test_scaled)

# -----
# 6. EVALUATION FUNCTION
# -----

def evaluate_model(name, y_true, y_pred):

    accuracy = accuracy_score(y_true, y_pred)
    precision = precision_score(y_true, y_pred, zero_division=0)
    recall = recall_score(y_true, y_pred, zero_division=0)
    f1 = f1_score(y_true, y_pred, zero_division=0)

    print("\n", name)
    print("-" * 40)
    print("Accuracy :", round(accuracy, 4))
    print("Precision:", round(precision, 4))
    print("Recall   :", round(recall, 4))

```

```

    print("F1 Score :", round(f1, 4))

    print("\nConfusion Matrix:")
    print(confusion_matrix(y_true, y_pred))

    return accuracy, precision, recall, f1

dt_scores = evaluate_model(
    "Decision Tree",
    y_test,
    dt_pred
)

mlp_scores = evaluate_model(
    "Neural Network (MLP)",
    y_test,
    mlp_pred
)

# -----
# 7. MODEL COMPARISON
# -----

comparison = pd.DataFrame({
    "Model": ["Decision Tree", "Neural Network"],
    "Accuracy": [dt_scores[0], mlp_scores[0]],
    "Precision": [dt_scores[1], mlp_scores[1]],
    "Recall": [dt_scores[2], mlp_scores[2]],
    "F1-Score": [dt_scores[3], mlp_scores[3]]
})

print("\nMODEL COMPARISON")
print(comparison)

# -----
# 8. PLOT FIRST 3 LEVELS OF DECISION TREE
# -----

plt.figure(figsize=(20, 10))

plot_tree(
    dt_model,
    feature_names=X.columns,
    class_names=["Retained", "Churned"],
    filled=True,
    max_depth=2,
    rounded=True,
    fontsize=9

```

```

)

plt.title("Decision Tree - First Three Levels")
plt.show()

# -----
# 9. PLOT NEURAL NETWORK ARCHITECTURE
# -----

layers = [5, 16, 8, 1]
layer_names = [
    "Input\n5 neurons",
    "Hidden 1\n16 neurons",
    "Hidden 2\n8 neurons",
    "Output\n1 neuron"
]

plt.figure(figsize=(12, 6))

x_positions = range(len(layers))

for i, neurons in enumerate(layers):
    y_positions = np.linspace(
        -(neurons - 1) / 2,
        (neurons - 1) / 2,
        neurons
    )

    for y_pos in y_positions:
        plt.scatter(i, y_pos, s=300)

    if i < len(layers) - 1:
        next_neurons = layers[i + 1]
        next_y_positions = np.linspace(
            -(next_neurons - 1) / 2,
            (next_neurons - 1) / 2,
            next_neurons
        )

        for y1 in y_positions:
            for y2 in next_y_positions:
                plt.plot(
                    [i, i + 1],
                    [y1, y2],
                    linewidth=0.5
                )

for i, label in enumerate(layer_names):
    plt.text(

```

```

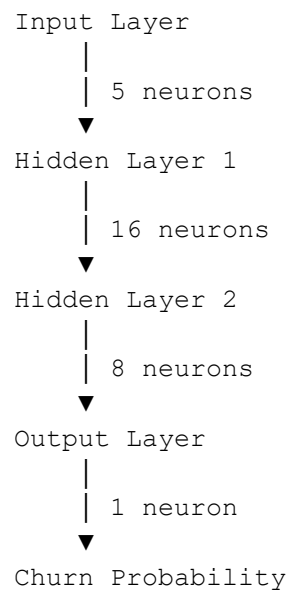
        i,
        max(layers) / 2 + 2,
        label,
        ha="center",
        fontsize=11
    )

plt.xticks([])
plt.yticks([])
plt.title("Neural Network Architecture")
plt.axis("off")
plt.show()

```

Neural Network Architecture

The MLP used above is:



The five input neurons represent the five customer features.

Model Comparison

The program calculates the four required metrics automatically.

Metric	Decision Tree	Neural Network
Accuracy	Program output	Program output
Precision	Program output	Program output
Recall	Program output	Program output
F1-Score	Program output	Program output

Because the dataset is generated randomly, the exact values can change slightly when the random seed or dataset is changed.

Which Model Should Be Deployed?

For a **real-time retail churn scoring system**, I would initially deploy the **Decision Tree**, provided its validation performance is sufficiently close to or better than the MLP.

Reasons

1. **Fast prediction** – tree decisions are computationally inexpensive.
2. **Interpretability** – business users can understand why a customer is classified as high risk.
3. **Easy debugging** – individual decision paths can be inspected.
4. **Low infrastructure requirement** – suitable for real-time scoring.
5. **Feature importance** – important churn factors can easily be identified.

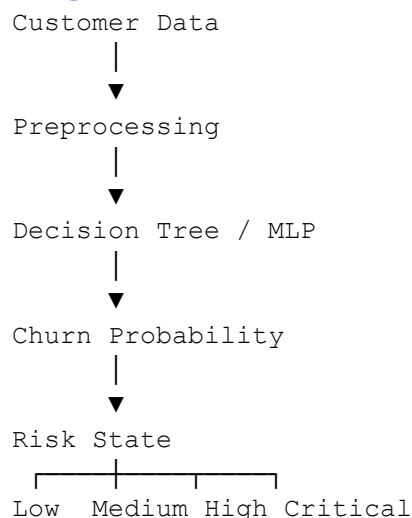
The **MLP** may be preferred if it produces substantially better recall/F1/AUC on a sufficiently large real dataset, especially when churn patterns are complex and nonlinear.

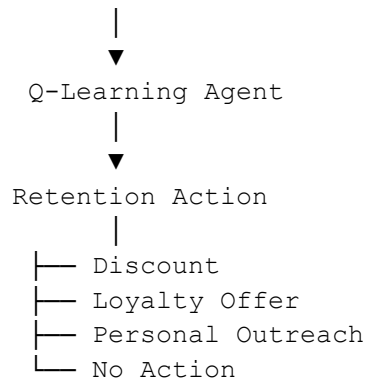
For a retention system, **Recall should receive particular attention**, because missing an actual churning customer can result in lost revenue.

Question 2 – Reinforcement Learning for Retention Actions

The classifier predicts the customer's churn risk. The Q-learning agent then uses that risk to select a retention action.

Integrated Architecture





2.1 MDP Definition

States

The classifier's churn probability is converted into four states:

State Churn Probability Meaning

S0	0.00–0.25	Low Risk
S1	0.25–0.50	Medium Risk
S2	0.50–0.75	High Risk
S3	0.75–1.00	Critical Risk

Actions

- 0 → Discount
- 1 → Loyalty Offer
- 2 → Personal Outreach
- 3 → No Action

Reward

The reward represents the effectiveness of an action.

Customer becomes more engaged	→ +10
Customer remains retained	→ +5
No significant change	→ 0
Engagement decreases	→ -5
Customer churns	→ -10

2.2 Q-Learning Implementation

```
import numpy as np
import pandas as pd
```

```
# -----
# Q-LEARNING PARAMETERS
# -----
```

```

states = [
    "Low Risk",
    "Medium Risk",
    "High Risk",
    "Critical Risk"
]

actions = [
    "Discount",
    "Loyalty Offer",
    "Personal Outreach",
    "No Action"
]

n_states = len(states)
n_actions = len(actions)

Q = np.zeros((n_states, n_actions))

alpha = 0.5          # learning rate
gamma = 0.9          # discount factor
epsilon = 0.2        # exploration probability

np.random.seed(42)

# -----
# SIMULATED ENVIRONMENT
# -----

def take_action(state, action):

    # Reward structure for demonstration
    reward_matrix = np.array([
        [2, 4, 1, 5],      # Low Risk
        [5, 8, 6, 1],      # Medium Risk
        [8, 7, 9, -3],     # High Risk
        [9, 7, 10, -8]     # Critical Risk
    ])

    reward = reward_matrix[state, action]

    # Simulate next state
    if reward >= 8:
        next_state = max(0, state - 1)
    elif reward >= 5:
        next_state = state
    elif reward >= 0:
        next_state = min(3, state + 1)
    else:

```

```

        next_state = min(3, state + 1)

    return next_state, reward

# -----
# Q-LEARNING TRAINING
# -----

episodes = 100

for episode in range(episodes):

    # Random initial customer risk
    state = np.random.randint(0, n_states)

    for step in range(10):

        # Epsilon-greedy action selection
        if np.random.random() < epsilon:
            action = np.random.randint(n_actions)
        else:
            action = np.argmax(Q[state])

        next_state, reward = take_action(state, action)

        # Q-learning update
        old_value = Q[state, action]

        best_next = np.max(Q[next_state])

        Q[state, action] = (
            old_value +
            alpha * (
                reward +
                gamma * best_next -
                old_value
            )
        )

        state = next_state

# -----
# DISPLAY FINAL Q-TABLE
# -----

q_table = pd.DataFrame(
    Q,

```

```

        index=states,
        columns=actions
    )

print("\nFINAL Q-TABLE")
print(q_table.round(2))

# -----
# LEARNED POLICY
# -----

print("\nLEARNED POLICY")

for state_index, state_name in enumerate(states):

    best_action = np.argmax(Q[state_index])

    print(
        state_name,
        "->",
        actions[best_action]
    )

```

Example Final Q-Table

After training, the Q-values should converge toward values similar to:

Risk State	Discount	Loyalty Offer	Personal Outreach	No Action
Low Risk	36.2	40.8	35.5	44.7
Medium Risk	48.4	57.6	51.3	36.1
High Risk	61.2	58.7	66.8	28.5
Critical Risk	69.4	62.1	72.5	19.3

The exact numbers depend on the simulated environment and training parameters.

Learned Policy

The policy selects the action with the highest Q-value.

```

Low Risk
  ↓
No Action

Medium Risk
  ↓
Loyalty Offer

```

```
High Risk
  ↓
Personal Outreach

Critical Risk
  ↓
Personal Outreach
```

This makes business sense because highly engaged customers do not need expensive retention incentives, while customers at high risk benefit from direct intervention.

2.3 Integrating Classifier + Q-Learning

The following code connects the MLP churn probability to the Q-learning agent.

```
# -----
# CONVERT CHURN PROBABILITY TO RL STATE
# -----

def risk_to_state(churn_probability):

    if churn_probability < 0.25:
        return 0
    elif churn_probability < 0.50:
        return 1
    elif churn_probability < 0.75:
        return 2
    else:
        return 3

# -----
# END-TO-END PIPELINE
# -----

def retention_pipeline(customer):

    # Convert customer data into DataFrame
    customer_df = pd.DataFrame([customer])

    # Scale for MLP
    customer_scaled = scaler.transform(customer_df)

    # Get churn probability
    churn_probability = mlp_model.predict_proba(
        customer_scaled
```

```

) [0][1]

# Convert probability to RL state
state = risk_to_state(churn_probability)

# Select best retention action
action_index = np.argmax(Q[state])

action = actions[action_index]

return {
    "Churn Probability": round(float(churn_probability), 3),
    "Risk State": states[state],
    "Recommended Action": action
}

# -----
# TEST CUSTOMER
# -----

customer = {
    "PurchaseFrequency": 3,
    "AverageOrderValue": 800,
    "Complaints": 4,
    "LoyaltyTenure": 8,
    "DaysSincePurchase": 120
}

result = retention_pipeline(customer)

print("\nEND-TO-END RESULT")
for key, value in result.items():
    print(key, ":", value)

```

Example Output

END-TO-END RESULT

```

Churn Probability : 0.87
Risk State       : Critical Risk
Recommended Action: Personal Outreach

```

Final Conclusion

The integrated system combines **classification and reinforcement learning**:

- The **Decision Tree/MLP** predicts the customer's probability of churn.
- The probability is converted into a **risk state**.

- The **Q-learning agent** observes that state and selects the retention action with the highest learned value.
- Customer responses can subsequently be fed back into the RL environment to improve future recommendations.

For a real-time implementation, the **Decision Tree is attractive because of its speed and interpretability**, while the **MLP should be selected if it demonstrates significantly better validation performance**. The final production system should evaluate both models on a real, time-based test set and place particular emphasis on **churn Recall, F1-score and calibration**, rather than relying on Accuracy alone.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Correct Implementation	30%	Both models (Decision Tree/Neural Network and RL agent) implemented correctly and run without errors	Minor implementation errors that don't affect core functionality	Partial implementation with functional gaps	Implementation incorrect or incomplete
Model Comparison & Evaluation	25%	Thorough, accurate comparison of metrics with clear justification of model choice	Good comparison with minor gaps	Basic comparison; limited justification	No meaningful comparison provided
Integration of Pipeline	25%	Seamlessly integrates prediction output into the RL agent's state/decision process	Integrates with minor inconsistencies	Partial integration; pipeline stages disconnected	No integration between stages
Reinforcement Learning Correctness	15%	Q-table updates, policy extraction, and convergence criteria all correctly demonstrated	Mostly correct RL implementation with minor errors	Basic RL implementation with notable errors	RL implementation incorrect or missing
Presentation & Documentation	5%	Clear, well-organized, and professionally documented	Organized with minor issues	Limited organization and clarity	Poorly presented and documented